Finance Manager Documentation - Nkotolane Pitso

Technology Stack

Language	Java 21
Framework	Spring Boot 3.2
Security	Spring Security 6
ORM	Spring Data JPA / Hibernate
Database	MySQL 8
Templating	Thymeleaf
Build Tool	Maven
Testing	JUnit 5 / AssertJ

1. Project Overview

Finance Manager is a full-stack web application for personal financial management. It allows users to track income and expenses, set monthly budgets by category, and generate visual reports. The application is built with Spring Boot and demonstrates enterprise software engineering practices including custom data structures, four design patterns, layered architecture, and Spring Security authentication.

This document is written for a developer who needs to understand, maintain, extend, or onboard to the project. It covers architecture, data structures, design patterns, security, database design, testing, and setup instructions.

1.1 Core Features

- User registration and login with BCrypt-hashed passwords
- Full transaction CRUD — create, view, edit, delete income and expense records
- Monthly budget limits per spending category with real-time alert checking
- Dashboard showing income, expenses, net balance, savings rate, and top categories
- Report generation with three interchangeable algorithms (Strategy pattern)
- Observer-driven budget alert system that fires on every transaction save
- Paginated transaction history with ownership-enforced security

1.2 Application Entry Points

URL	Description
/auth/login	Login page (public)
/auth/register	Registration page (public)
/dashboard	Main dashboard (authenticated)

/transactions	Paginated transaction list
/transactions/new	Add transaction form
/transactions/{id}/edit	Edit transaction form
/reports	Report generation page

2. Architecture

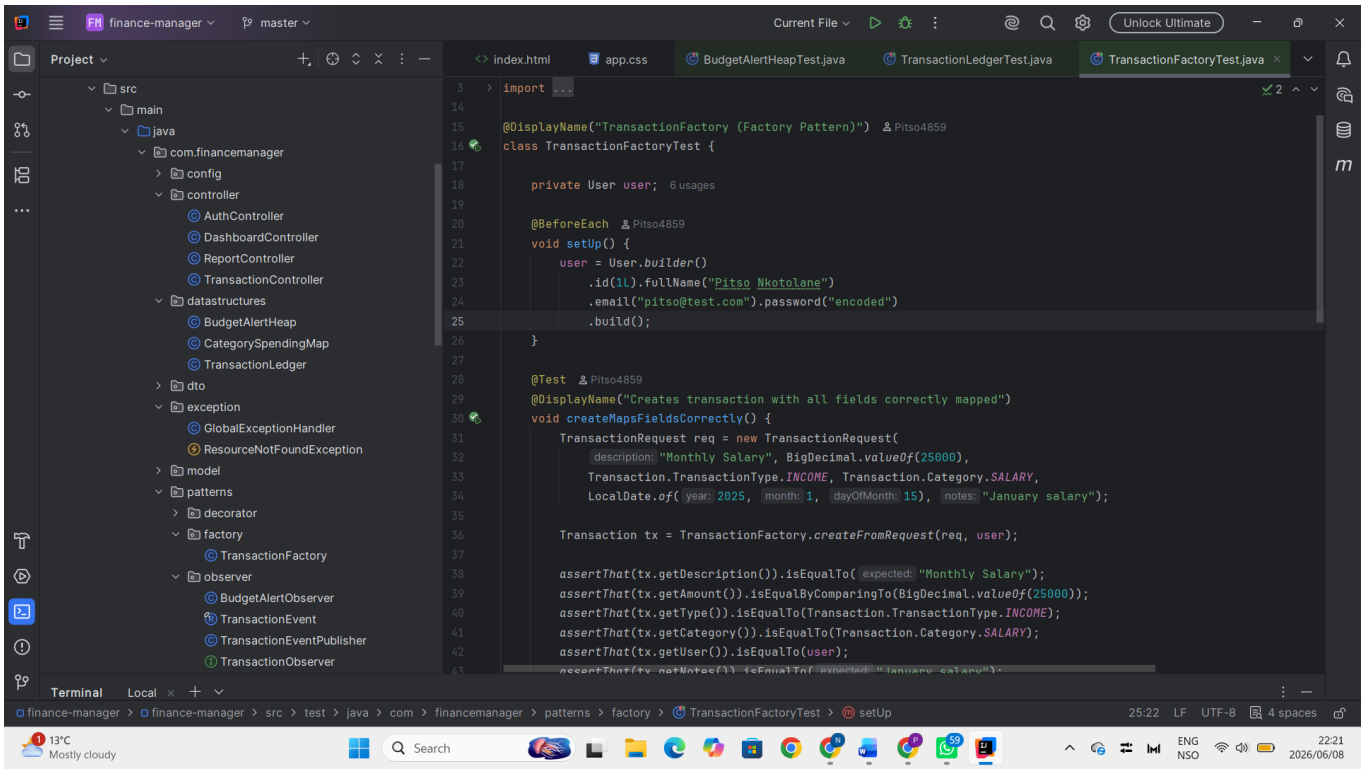
2.1 Layered Architecture

The application uses a strict three-layer architecture. Each layer has a single responsibility and communicates only with the layer directly below it.

Layer	Package	Responsibility
Presentation	controller/	Handle HTTP requests, bind form data, select views. No business logic.
Business Logic	service/	Enforce rules, coordinate data structures and patterns, manage transactions.
Data Access	repository/	Database queries via Spring Data JPA. Persistence only.

Key Principle: Controllers never call repositories directly. Services never reach into the HTTP layer. This makes each layer independently testable and replaceable.

2.2 Package Structure



2.3 Request Flow

The following shows the complete lifecycle of a POST /transactions request (create transaction):

1. Browser submits the transaction form as HTTP POST.
2. TransactionController receives the request, validates @Valid TransactionRequest DTO.
3. If validation passes, controller calls TransactionService.create(request, user).
4. TransactionServiceImpl calls TransactionFactory.createFromRequest() which validates category-type consistency and builds the entity.
5. The entity is persisted via TransactionRepository.save().
6. TransactionEventPublisher.publish() fires a TransactionEvent.CREATED.
7. BudgetAlertObserver receives the event, checks the user's budget, logs a warning if over 80%.
8. Controller issues a redirect (PRG pattern) to prevent duplicate form submission.
9. Browser follows redirect to GET /transactions, displaying a success flash message.

3. Custom Data Structures

Three custom data structures were built from scratch rather than relying on Java's standard library. Each was chosen because of a specific performance or structural requirement of the application.

3.1 TransactionLedger — Doubly Linked List

File: datastructures/TransactionLedger.java

Problem it solves

Transaction history must be traversed in two directions: oldest to newest for reports and balance calculation, newest to oldest for recent activity views. An ArrayList requires O(n) shifting on inserts and has resize overhead. A singly linked list cannot traverse backwards.

Why a doubly linked list

- O(1) prepend via addFirst() — newest transaction always at head without shifting
- O(1) append via addLast() — for batch loading from the database
- Bidirectional iteration — forward (oldest to newest) and reverse (newest to oldest) in a single pass
- No array resizing — memory is allocated per node only when needed

Internal structure

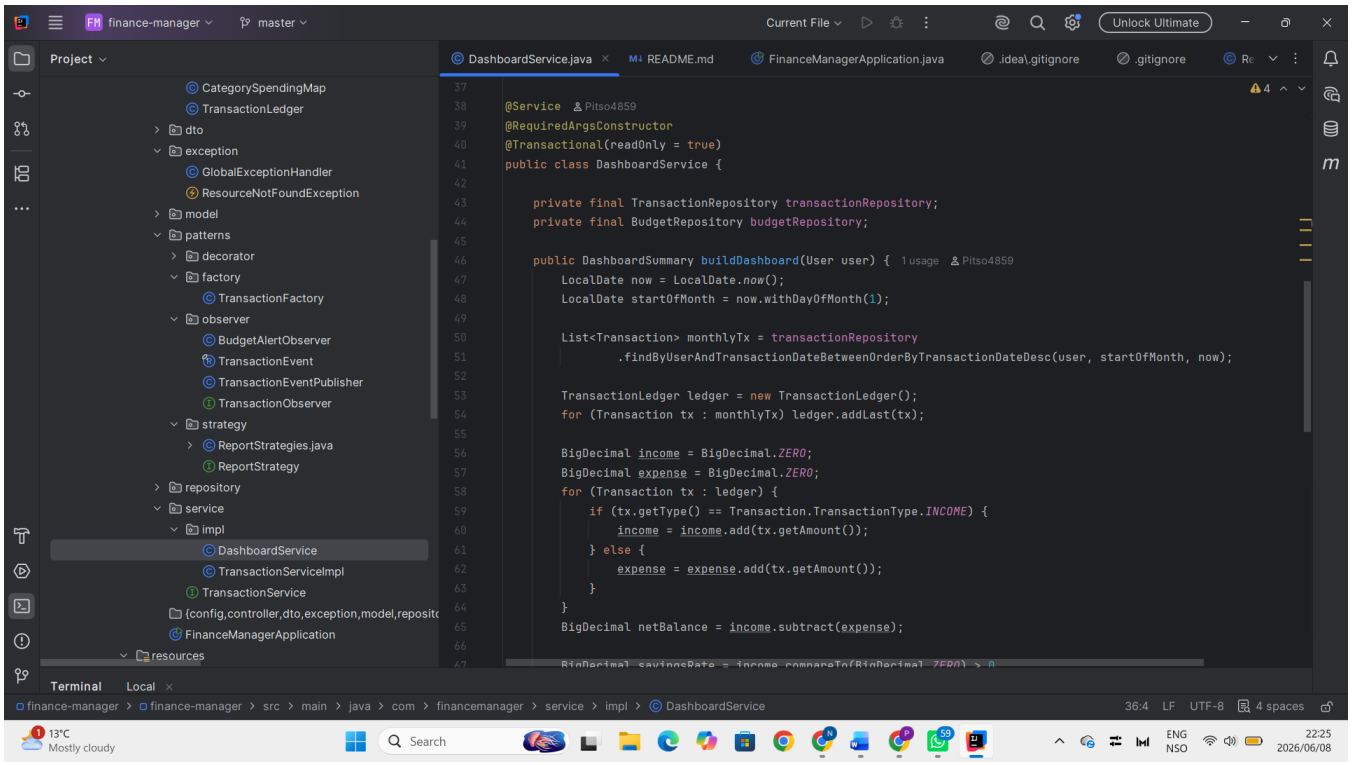
```
head <--> Node(tx3) <--> Node(tx2) <--> Node(tx1) <--> tail

Each Node holds:
  Transaction data
  Node prev  (pointer to previous node)
  Node next  (pointer to next node)
```

Time Complexity

Method	Complexity	Reason
addFirst()	O(1)	Only head pointer changes
addLast()	O(1)	Only tail pointer changes
removeById(id)	O(n)	Linear scan to find node by id
computeBalance()	O(n)	Must visit all nodes
totalByCategory()	O(n)	Must visit all nodes
size()	O(1)	Counter maintained on every add/remove
peekFirst() / peekLast()	O(1)	Direct head/tail pointer access

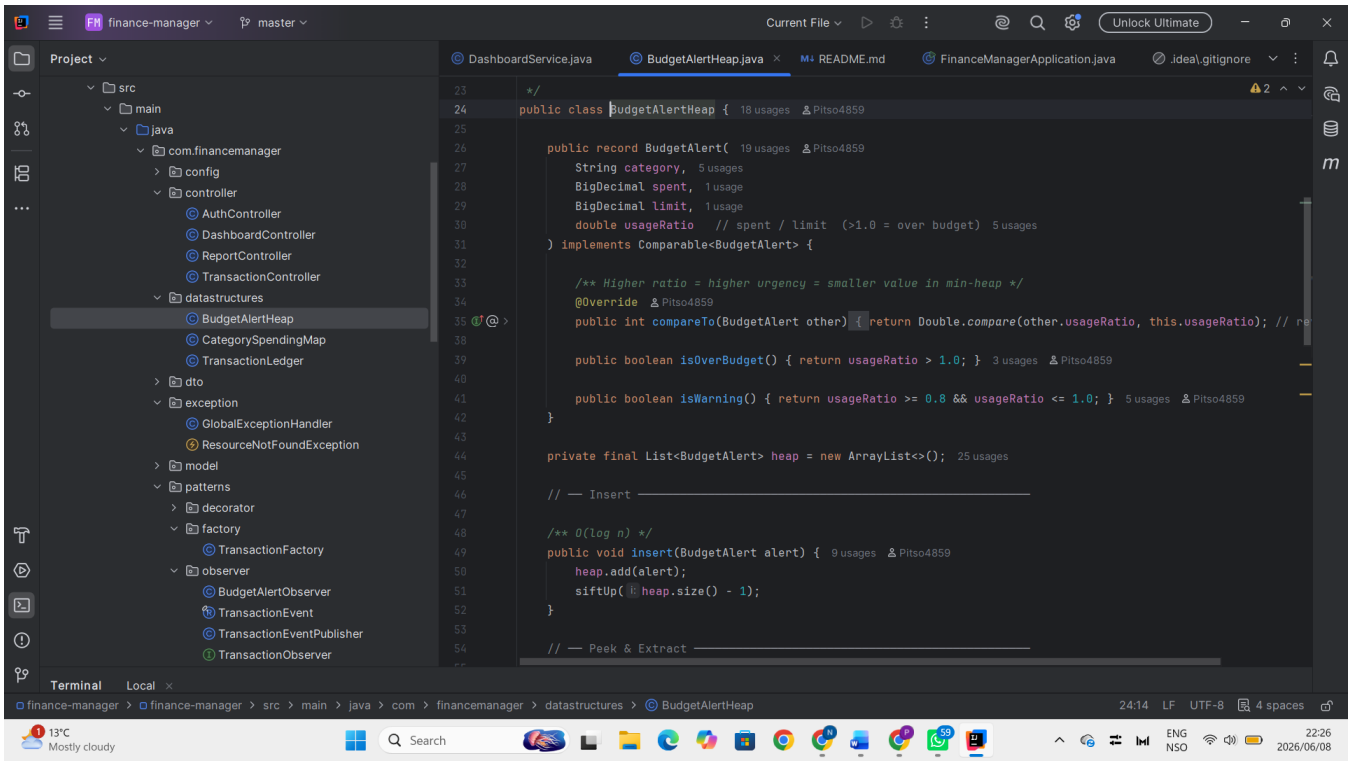
Usage in the application

DashboardService loads this month's transactions into a TransactionLedger, then traverses it once to sum income and expenses into the running balance. The reverse iterator is used for "recent activity" views.



3.2 BudgetAlertHeap — Min-Heap

File: datastructures/BudgetAlertHeap.java



Problem it solves

After saving a transaction, the dashboard must show the most urgent budget alerts at the top — categories that are closest to or over their monthly limit. Sorting all categories every time the dashboard loads is O(k log k). We need the most urgent category in O(1).

Why a min-heap

Priority is defined as the overspend ratio: spent / limit. A ratio above 1.0 means the category is over budget. The heap orders by inverse ratio so the most over-budget category sits at the root (index 0) and can be read in O(1). New alerts insert in O(log k) which is far cheaper than re-sorting.

Internal structure

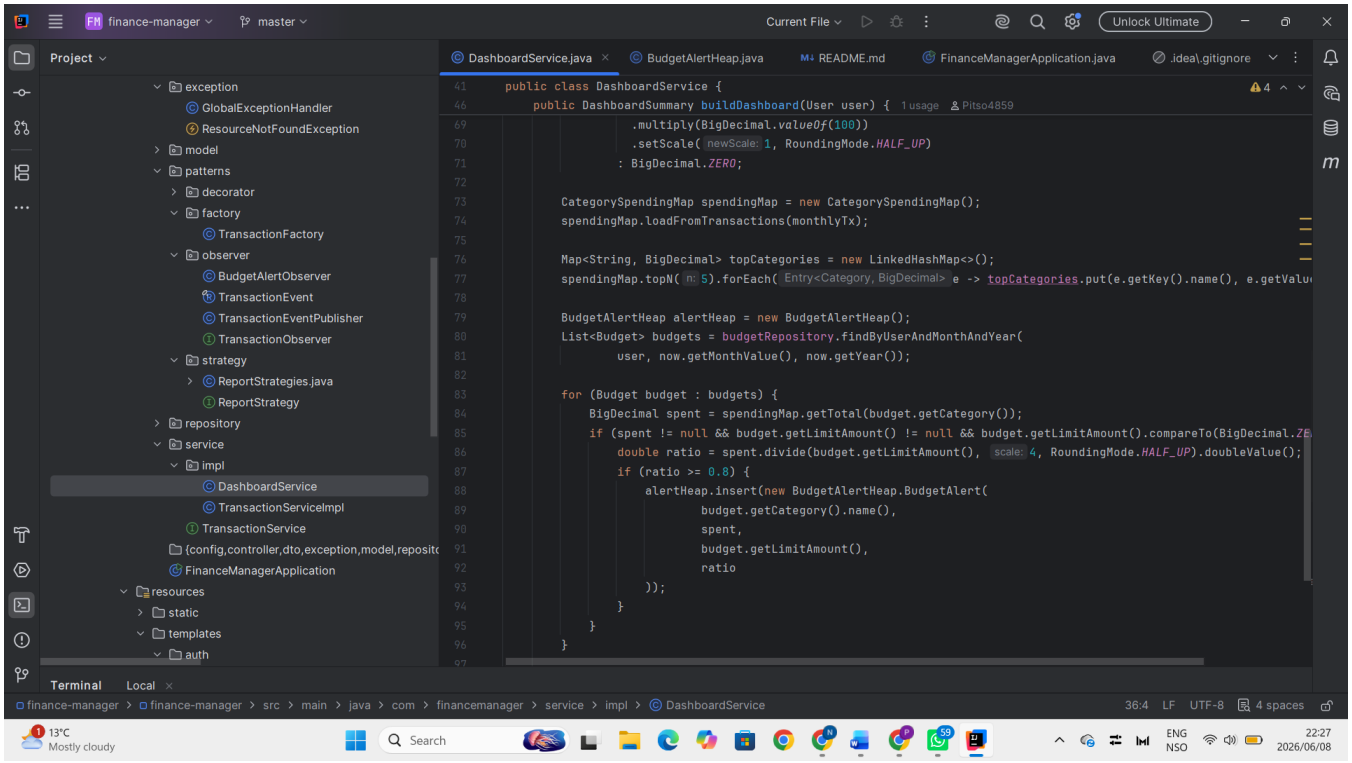
```
Array-backed binary heap:

  parent(i)      = (i - 1) / 2
  left child(i)  = 2 * i + 1
  right child(i) = 2 * i + 2

Example (ratio = spent / limit):

          HOUSING (1.30)
         /              \
  TRANSPORT (1.15)    FOOD (0.82)
  /        \
ENTERTAIN(0.91)  ...
```

Time Complexity

Method	Complexity	Reason
insert(alert)	O(log k)	Sift up after append
peek()	O(1)	Root is always most urgent
extractTop()	O(log k)	Swap root with last, sift down
drainAll()	O(k log k)	Repeated extractTop()

Usage in the application

DashboardService iterates the user's budgets, computes a usage ratio for each one, inserts only those at 80% or higher into the heap, then drains the heap to get an urgency-sorted list for the dashboard template.



3.3 CategorySpendingMap — EnumMap

File: datastructures/CategorySpendingMap.java

Problem it solves

The dashboard needs to aggregate total spending per category (e.g. "how much did the user spend on FOOD this month?") and find the top 5 categories. HashMap<Category, BigDecimal> works, but incurs hash computation and potential collision overhead on every access.

Why EnumMap

EnumMap uses the enum ordinal as an array index. get() and put() are direct array lookups — no hashing, no collision resolution, no Entry object allocation per key. Iteration order is the enum declaration order, which is stable and predictable.

Time Complexity

Method	Complexity	Reason
accumulate(category, amount)	O(1)	Direct array index via ordinal
getTotal(category)	O(1)	Direct array index via ordinal
getPercentage(category)	O(1)	Division on two cached values
topN(n)	O(k log k)	Stream sort where k = total categories
loadFromTransactions(list)	O(n)	Linear scan of transaction list

Usage in the application

```
CategorySpendingMap spendingMap = new CategorySpendingMap();
spendingMap.loadFromTransactions(monthlyTx);    // O(n) — one pass

// O(1) lookups for individual categories
BigDecimal foodSpend = spendingMap.getTotal(Category.FOOD);
BigDecimal foodPct   = spendingMap.getPercentage(Category.FOOD);

// O(k log k) — top 5 for the dashboard chart
List<Map.Entry<Category, BigDecimal>> top5 = spendingMap.topN(5);
```

4. Design Patterns

Four design patterns are implemented. Each one solves a concrete problem in the codebase — they are not applied for academic show.

4.1 Observer Pattern — Budget Alert Event System

Files: patterns/observer/

Problem

When a transaction is saved, multiple subsystems must react: check the user's budget and log a warning, write to an audit log, and potentially send a notification email. Wiring all of this directly inside TransactionServiceImpl would violate the Single Responsibility Principle and create tight coupling. Adding a new side effect would require modifying a tested, working service.

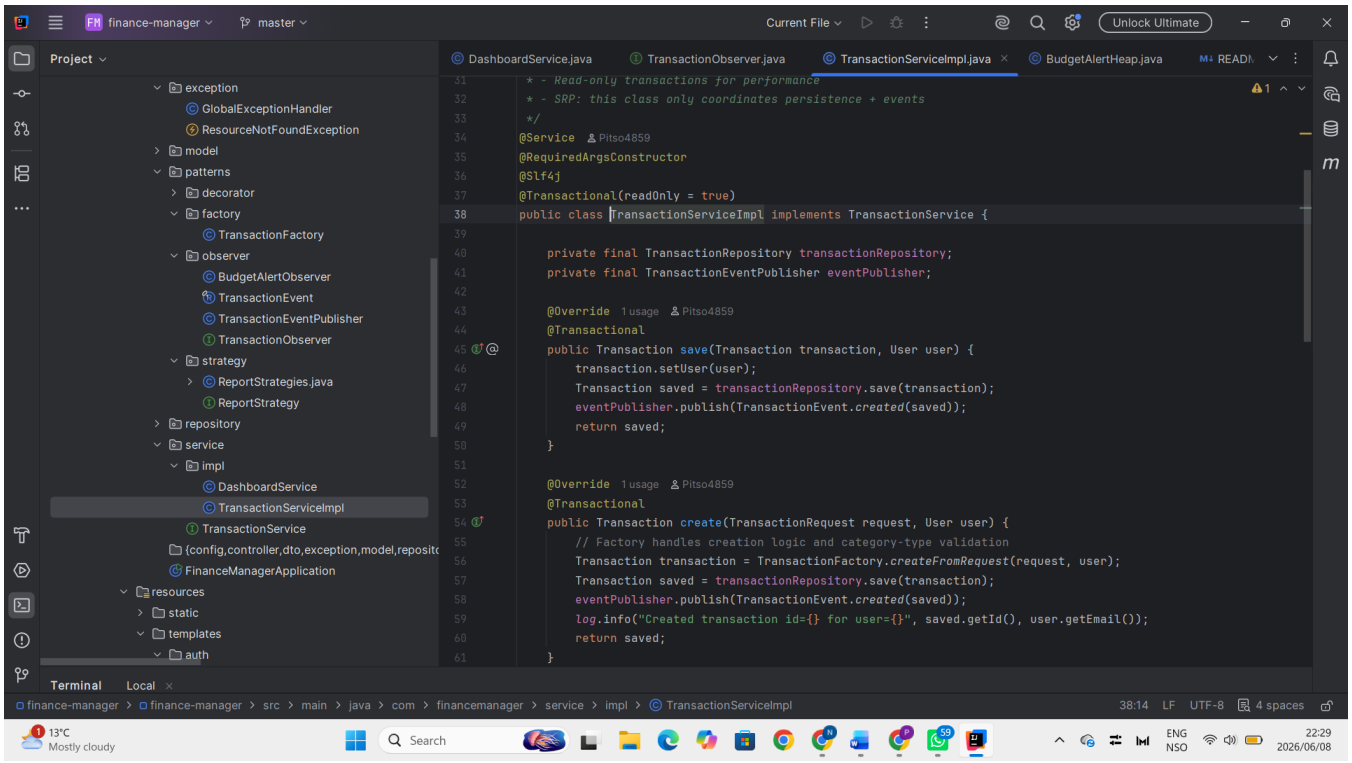
Solution

The Observer pattern decouples the transaction save from its side effects. TransactionServiceImpl publishes an event and knows nothing about who handles it. Observers subscribe at startup and react independently.

Participant	Role
TransactionEvent	Immutable record carrying transaction + event type (CREATED / UPDATED / DELETED)
TransactionObserver	Interface — any class implementing onTransactionEvent(event) can subscribe
TransactionEventPublisher	The Subject — maintains observer list, publishes events with defensive try-catch per observer
BudgetAlertObserver	Concrete observer — checks budget limits and logs warnings when triggered

How to add a new side effect

Create a new class implementing TransactionObserver, annotate it @Component, and call publisher.subscribe(this) in @PostConstruct. TransactionServiceImpl does not change at all.



4.2 Strategy Pattern — Report Generation

Files: patterns/strategy/

Problem

The application needs to produce three different report types, each with different aggregation logic. Without a pattern, ReportController would contain a large if/else or switch block. Adding a fourth report type requires editing existing, working code — a violation of the Open/Closed Principle.

Solution

Each algorithm implements the ReportStrategy interface. ReportController holds a Map<String, ReportStrategy> injected by Spring, selects the right strategy at runtime based on the user's selection, and calls generate(). No if/else anywhere in the controller.

Strategy	Algorithm
MonthlySummaryStrategy	Groups transactions by month, produces income vs expense bar chart data
CategoryBreakdownStrategy	Uses CategorySpendingMap for O(1) aggregation, produces pie chart data
TrendAnalysisStrategy	Computes 3-month rolling average of expenses, produces line chart data

How to add a new report type

Create a new class implementing ReportStrategy, annotate it @Component. Spring automatically adds it to the injected strategies map. ReportController picks it up with no changes.

```
// Step 1: implement the interface
@Component
class YearlyComparisonStrategy implements ReportStrategy {
    @Override public String getName() { return "Yearly Comparison"; }
    @Override public ReportResult generate(List<Transaction> transactions) {
        // your algorithm here
    }
}

// Step 2: nothing else. The new strategy appears in the UI dropdown automatically.
```

4.3 Factory Method Pattern — Transaction Creation

File: patterns/factory/TransactionFactory.java

Problem

Creating a Transaction entity involves more than simple field assignment: descriptions must be trimmed, timestamps injected, the owner assigned, and a business rule enforced — income categories cannot appear on expense transactions and vice versa. Doing all of this inline in the controller or service scatters creation logic and makes it difficult to test.

Solution

TransactionFactory is a static factory class that centralises all creation and update logic. It enforces the category-type business rule using a pattern-matching switch expression (Java 21). Controllers and services call one line. Testing the creation rule requires only a factory unit test.

Business rule enforced at creation time

```
boolean isIncomeCategory = switch (request.category()) {
    case SALARY, FREELANCE, INVESTMENT, GIFT, OTHER_INCOME -> true;
    default -> false;
};

if (isIncomeCategory && isExpenseType)
    throw new IllegalArgumentException("Category SALARY cannot be EXPENSE.");

if (!isIncomeCategory && !isExpenseType)
    throw new IllegalArgumentException("Category FOOD cannot be INCOME.");
```

4.4 Decorator Pattern — Validated Transaction Service

File: patterns/decorator/ValidatedTransactionService.java

Problem

Business-level validation rules need to be enforced on transactions: amounts must be positive, must not exceed R1,000,000, dates cannot be in the future, and descriptions cannot be blank. Adding these checks directly to TransactionServiceImpl mixes concerns and makes the service harder to read and test.

Solution

ValidatedTransactionService wraps a TransactionService delegate. It runs validation before calling the real service. This is identical in concept to Java's BufferedReader wrapping FileReader to add buffering without changing the FileReader. The real service stays clean. New validation rules are added to the decorator only.

```
public class ValidatedTransactionService {
    private final TransactionService delegate;  // the real service

    public Transaction save(Transaction tx, User user) {
        preValidate(tx);                    // validate first
        return delegate.save(tx, user);         // then delegate
    }
}
```

5. Domain Model

5.1 Entities

User

Field	Description
id	Long — auto-generated primary key
fullName	String — display name
email	String — unique, used as login username
password	String — BCrypt hashed, cost factor 12
role	Enum — USER or ADMIN
enabled	boolean — soft disable without deletion
createdAt	LocalDateTime — set once on insert, never updated
transactions	List<Transaction> — lazy-loaded, cascade ALL
budgets	List<Budget> — lazy-loaded, cascade ALL

Transaction

Field	Description
id	Long — auto-generated primary key

description	String — required, trimmed at factory level
amount	BigDecimal(15,2) — never float or double
type	Enum — INCOME or EXPENSE
category	Enum — 5 income + 10 expense categories
transactionDate	LocalDate — past-or-present enforced by validation
notes	String — optional, max 500 chars
createdAt	LocalDateTime — audit field, never updated
user	ManyToOne — owner, indexed for query performance

Why BigDecimal?
float and double are binary floating-point types. They cannot represent 0.10 exactly in binary, causing rounding errors that accumulate over financial calculations. BigDecimal uses decimal arithmetic with exact precision. Always use BigDecimal for money.

Budget

Field	Description
id	Long — auto-generated primary key
category	Enum — matches Transaction.Category
limitAmount	BigDecimal — monthly spending limit for this category
month	Integer — 1 to 12
year	Integer — full year e.g. 2025
user	ManyToOne — owner

A unique constraint prevents duplicate budgets: (user_id, category, month, year) must be unique. One budget per category per month per user.

5.2 Database Indexes

Index	Purpose
idx_user_email	Unique index on users.email — fast login lookup
idx_tx_user_date	Composite index on (user_id, transaction_date) — fast date range queries
idx_tx_category	Index on transactions.category — fast category aggregation
uq_budget_user_category_month_year	Unique constraint preventing duplicate budgets

6. Security

6.1 Authentication

Spring Security 6 is configured in SecurityConfig. Authentication uses DaoAuthenticationProvider backed by UserDetailsService, which loads users from the database by email. Passwords are hashed with BCryptPasswordEncoder at cost factor 12.

- **Cost factor 12** means BCrypt performs 2^12 = 4096 hashing iterations. This makes brute-force attacks computationally expensive.
- Passwords are never stored in plain text at any point in the system.
- Sessions are invalidated on logout and the JSESSIONID cookie is deleted.
- Remember-me tokens are valid for 7 days (604,800 seconds).

6.2 Authorisation

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**", "/css/**", "/js/**").permitAll()  // public
    .requestMatchers("/admin/**").hasRole("ADMIN")                 // admin only
    .anyRequest().authenticated()                                  // everything else: login required
)
```

> **Ownership Enforcement**
> Every read and write operation in TransactionService uses findByIdAndUser(id, user). This means a user cannot view, edit, or delete a transaction that belongs to another user, even if they know the transaction ID. This is checked in the service layer, not the controller.

6.3 CSRF Protection

Spring Security enables CSRF protection by default. All state-changing forms (POST, DELETE) include a hidden CSRF token injected by Thymeleaf using th:name="${_csrf.parameterName}" and th:value="${_csrf.token}". Requests without a valid token are rejected with 403.

6.4 Route Summary

Pattern	Access
/auth/**	Public — no authentication required
/css/**, /js/**	Public — static assets
/admin/**	ADMIN role required
All other routes	Authentication required (any role)

7. Service Layer

7.1 TransactionService Interface

Controllers depend on the TransactionService interface, not the concrete implementation. This is the Dependency Inversion Principle. The interface defines the contract; TransactionServiceImpl provides the implementation. To swap to a different implementation (e.g. in-memory for testing), no controller code changes.

Method	Behaviour
create(request, user)	Creates a new transaction via factory, fires CREATED event
update(id, request, user)	Validates ownership, updates via factory, fires UPDATED event
delete(id, user)	Validates ownership, deletes, fires DELETED event
findByIdAndUser(id, user)	Throws ResourceNotFoundException if not found or not owned
findPagedByUser(user, pageable)	Returns a paginated Page<Transaction>
findByUserAndDateRange(user, start, end)	Used by report generation
sumByUserTypeAndPeriod(...)	Aggregate query for dashboard totals
findRecentByUser(user)	Returns last 5 transactions for dashboard widget

7.2 Transaction Management

All write operations (create, update, delete) are annotated @Transactional. If any step fails — persistence, event publishing, or anything else — the database transaction rolls back and no partial state is written.

The class-level annotation @Transactional(readOnly = true) is applied to TransactionServiceImpl as a default for all read methods. This tells Hibernate to skip dirty-checking on reads, which improves performance. Write methods override this with @Transactional.

7.3 DashboardService

DashboardService.buildDashboard() orchestrates all three custom data structures in a single method call. The sequence is:

10. Load this month's transactions from the database via TransactionRepository.
11. Load them into a TransactionLedger (doubly linked list) for in-memory traversal.
12. Traverse the ledger once to compute total income and total expense.
13. Load the same transactions into CategorySpendingMap for O(1) category lookups.

14. Query the user's budgets for this month, compute usage ratios, insert warnings into BudgetAlertHeap.
15. Drain the heap to get urgency-sorted budget alerts.
16. Assemble all values into a DashboardSummary record and return.

8. Repository Layer

Repositories are Spring Data JPA interfaces. Spring generates all SQL at startup from the method names and @Query annotations. No JDBC boilerplate is written by hand.

8.1 TransactionRepository

Method	Purpose
findByIdAndUser(id, user)	Ownership-safe lookup. Returns Optional.empty() instead of throwing if not found.
findByUserOrderByTransactionDateDesc(user, pageable)	Paginated list, newest first.
findByUserAndTransactionDateBetween(...)	Date range query for reports. Uses composite index.
sumByUserAndTypeAndDateRange(...)	@Query with COALESCE — returns 0 instead of null when no rows match.
sumExpensesByCategoryAndMonthAndYear(...)	Used by BudgetAlertObserver after a transaction is saved.
findTop5ByUserOrderByTransactionDateDesc(user)	Dashboard recent activity widget.

8.2 JPQL Queries

Two methods use custom @Query with JPQL (Java Persistence Query Language) rather than derived method names because they require aggregation (SUM) and month/year extraction functions:

```
@Query("SELECT COALESCE(SUM(t.amount), 0) FROM Transaction t " +
       "WHERE t.user = :user AND t.type = :type " +
       "AND t.transactionDate BETWEEN :start AND :end")
BigDecimal sumByUserAndTypeAndDateRange(
    @Param("user") User user,
    @Param("type") TransactionType type,
    @Param("start") LocalDate start,
    @Param("end") LocalDate end
);
```

> **COALESCE**
> Without COALESCE, SUM returns NULL when no rows match the filter (e.g. a new user with no transactions). COALESCE(SUM(...), 0) converts NULL to 0, so the service layer never needs to null-check the return value from aggregate queries.

9. Controllers

Controllers are intentionally thin. They handle only HTTP concerns: receiving requests, binding/validating data, calling the appropriate service method, and selecting a view or redirect. No business logic lives in controllers.

9.1 Pattern: Post-Redirect-Get

Every form submission (POST) that succeeds redirects to a GET endpoint. This prevents duplicate form submission if the user refreshes the browser after a successful save. Flash attributes (RedirectAttributes.addFlashAttribute) carry success or error messages across the redirect.

```
@PostMapping
public String create(@Valid @ModelAttribute TransactionRequest request,
                     BindingResult result, ...,
                     RedirectAttributes redirectAttributes) {
    if (result.hasErrors()) return "transaction/form";  // re-render with errors

    transactionService.create(request, user);
    redirectAttributes.addFlashAttribute("successMessage", "Transaction added.");
    return "redirect:/transactions";  // PRG: redirect, not forward
}
```

9.2 Form Validation

TransactionRequest is a Java record annotated with Bean Validation constraints (@NotBlank, @DecimalMin, @PastOrPresent, etc.). The controller uses @Valid to trigger validation. If any field fails, BindingResult captures the errors and the controller re-renders the form with error messages — no redirect.

9.3 Controller Summary

Controller	Endpoints
AuthController	GET /auth/login, GET /auth/register, POST /auth/register
DashboardController	GET /dashboard — calls DashboardService, renders summary
TransactionController	GET/POST /transactions, GET/POST /transactions/{id}/edit, POST /transactions/{id}/delete
ReportController	GET /reports — selects strategy at runtime, renders chart data

10. Thymeleaf Templates

10.1 Template Structure

Template	Purpose
fragments/layout.html	Shared head, sidebar/navbar, alert and scripts fragments
auth/login.html	Login form with remember-me and flash messages
auth/register.html	Registration form with field-level validation errors
dashboard/index.html	Summary cards, Chart.js doughnut, budget alerts, recent transactions
transaction/list.html	Paginated table with edit and delete actions
transaction/form.html	Create and edit form — reused via transactionId model attribute
report/index.html	Strategy selector dropdown, dynamic Chart.js rendering
error/400.html, 404.html, 500.html	Friendly error pages rendered by GlobalExceptionHandler

10.2 Fragment Reuse

All pages include shared fragments from layout.html using Thymeleaf's th:replace syntax. This ensures the navbar, fonts, CSS, and Bootstrap JS are included consistently without duplication.

```
<head th:replace="~{fragments/layout :: head('Dashboard')}"></head>
<nav th:replace="~{fragments/layout :: navbar}"></nav>
<div th:replace="~{fragments/layout :: alerts}"></div>
<div th:replace="~{fragments/layout :: scripts}"></div>
```

10.3 Security in Templates

The Thymeleaf Spring Security extras library provides sec:authentication and sec:authorize tags. The navbar uses these to display the logged-in user's name and render role-specific menu items.

```
<span sec:authentication="name"></span>        <!-- displays logged-in email -->
<div sec:authorize="hasRole('ADMIN')">         <!-- only rendered for ADMIN -->
    <a href="/admin">Admin Panel</a>
</div>
```

11. Exception Handling

GlobalExceptionHandler is annotated @ControllerAdvice and intercepts unhandled exceptions from any controller. It maps exception types to view names and adds a user-friendly error message to the model. Stack traces are never exposed to the browser.

Exception Type	Handling
ResourceNotFoundException	Thrown when a transaction is not found or ownership check fails. Renders error/404.html.
IllegalArgumentException	Thrown by TransactionFactory on invalid category-type combination. Renders error/400.html.
Exception (catch-all)	Any unexpected error. Renders error/500.html with a generic message.

> **Logging**
> GlobalExceptionHandler logs WARN for 400/404 and ERROR for 500. This ensures unexpected exceptions are visible in production logs without showing details to the user.

12. Testing

Unit tests cover the three custom data structures and the factory pattern. Tests use JUnit 5 and AssertJ. No Spring context is loaded — these are pure Java unit tests that run in milliseconds.

12.1 Test Files

Test Class	Coverage
TransactionLedgerTest	7 tests — size, order, balance computation, removeById, reverse iteration
BudgetAlertHeapTest	5 tests — empty peek, priority ordering, drainAll sort, isOverBudget, isWarning
TransactionFactoryTest	4 tests — field mapping, whitespace trimming, invalid category-type combos

12.2 Running Tests

```
mvn test

# Run a specific test class
mvn test -Dtest=TransactionLedgerTest

# Run with verbose output
mvn test -Dsurefire.failIfNoSpecifiedTests=false
```

12.3 Test Examples

Data structure correctness

```
@Test
@DisplayName("computeBalance correctly sums income minus expenses")
void computeBalanceIsCorrect() {
    ledger.addLast(tx(1L, "Salary",    valueOf(10000), INCOME));
    ledger.addLast(tx(2L, "Rent",      valueOf(4000),  EXPENSE));
    ledger.addLast(tx(3L, "Groceries", valueOf(1500),  EXPENSE));

    // 10000 - 4000 - 1500 = 4500
    assertThat(ledger.computeBalance()).isEqualByComparingTo(valueOf(4500));
}
```

Business rule enforcement

```
@Test
@DisplayName("Throws when income category assigned to EXPENSE type")
```

```
void throwsOnIncomeCategoryWithExpenseType() {
    TransactionRequest req = new TransactionRequest(
        "Bad", valueOf(100), EXPENSE, Category.SALARY, LocalDate.now(), null);

    assertThatThrownBy(() -> TransactionFactory.createFromRequest(req, user))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("SALARY")
        .hasMessageContaining("EXPENSE");
}
```

13. Local Setup

13.1 Prerequisites

Requirement	Notes
Java	21 or higher — check with: java -version
Maven	3.9 or higher — check with: mvn -version
MySQL	8.0 or higher — must be running locally
Git	Any recent version

13.2 Setup Steps

17. Clone the repository

```
git clone https://github.com/Pitso4859/finance-manager.git
cd finance-manager
```

18. Create the database

```
mysql -u root -p -e "CREATE DATABASE finance_manager;"
```

19. Configure your credentials in src/main/resources/application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/finance_manager?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=your_password_here
```

20. Run the application

```
mvn spring-boot:run
```

21. Open in browser

```
http://localhost:8080/auth/register
```

13.3 Configuration Reference

Property	Notes
spring.jpa.hibernate.ddl-auto=update	Hibernate auto-creates/updates tables on startup. Change to validate in production.
spring.thymeleaf.cache=false	Templates reload on every request. Set to true in production.

logging.level.com.financemanager=DEBUG	Verbose logging for development. Set to INFO or WARN in production.
spring.jpa.show-sql=false	Set to true during development to see generated SQL in the console.

14. SOLID Principles Reference

The following table maps each SOLID principle to where it is applied in the codebase.

Principle	Applied In
Single Responsibility	Controllers handle only HTTP. Services handle only business logic. Factories handle only creation. Repositories handle only persistence.
Open / Closed	New report types extend ReportStrategy without modifying ReportController. New side effects extend TransactionObserver without modifying TransactionServiceImpl.
Liskov Substitution	Any class implementing TransactionService can replace TransactionServiceImpl. Controllers are not affected.
Interface Segregation	TransactionService defines only the methods controllers need. No fat interfaces with unrelated operations.
Dependency Inversion	TransactionController depends on TransactionService interface, not TransactionServiceImpl. Spring injects the concrete class at runtime.

15. How to Extend the Application

Adding a New Report Type

22. Create a new class implementing ReportStrategy in the patterns/strategy/ package.
23. Annotate it @Component.
24. Implement getName() and generate(List<Transaction> transactions).

That is all. Spring auto-discovers the component, adds it to the strategies map, and it appears in the report type dropdown without any other changes.

Adding a New Transaction Side Effect

25. Create a class implementing TransactionObserver.
26. Annotate it @Component and inject TransactionEventPublisher.
27. Call publisher.subscribe(this) in a @PostConstruct method.
28. Implement onTransactionEvent(TransactionEvent event) with your logic.

TransactionServiceImpl does not change.

Adding a New Transaction Category

29. Add the new value to Transaction.Category enum.
30. Determine if it is an income or expense category.
31. Update the switch expression in TransactionFactory.validateCategoryType() to include the new value in the correct case.
32. Add the new option to the category select in transaction/form.html.

Adding a New Entity

33. Create the entity class in model/ with @Entity and appropriate JPA annotations.
34. Create a repository interface in repository/ extending JpaRepository.
35. Create a service interface and implementation in service/ and service/impl/.
36. Create a controller in controller/ with the required request mappings.
37. Create Thymeleaf templates in templates/your-entity/.
38. Add navigation links to fragments/layout.html.

16. Known Limitations and Future Work

Limitation	Notes
No Budget CRUD in UI	Budgets are defined in the data model and checked by the observer, but there is no browser interface to create or manage budgets. A BudgetController and budget templates would complete this.
No email notifications	BudgetAlertObserver logs warnings but does not send emails. The mail dependency is in pom.xml and the SMTP config is commented in application.properties — wiring these together completes the feature.
No CSV import	Transactions are entered one at a time. A batch import endpoint accepting CSV would use addLast() on the TransactionLedger for O(1) sequential loading.
In-memory data structures	TransactionLedger and BudgetAlertHeap are rebuilt per request. For very large transaction histories, a caching layer (e.g. Spring Cache + Redis) could persist them between requests.
Session-based auth only	The application uses form login with HTTP sessions. A REST/JWT layer would be needed to support a mobile client or a separate frontend framework.